DATACLEANAI: AUTOMATED DATA QUALITY, CLEANING & VALIDATION PLATFORM

A Project Report Submitted in

the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Yakkala Guna Veera Sri Amith – 2420030234

Kotte Manoj – 2420030245

BVS Jayadeep – 2420030381

Under the Esteemed Guidance of

Dr. Bonagani Prathusha

Department of Computer Science and Engineering

AY-2026–2027

Semester: Odd

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**DataCleanAI: Automated Data Quality, Cleaning & Validation Platform**" is a record of Bonafide work of **Yakkala Guna Veera Sri Amith – 2420030234, Kotte Manoj – 2420030245, BVS Jayadeep – 2420030381** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Yakkala Guna Veera Sri Amith – 2420030234

Kotte Manoj – 2420030245

BVS Jayadeep – 2420030381

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled "**DataCleanAI: Automated Data Quality, Cleaning & Validation Platform**" is a bonafide work done and submitted by **Yakkala Guna Veera Sri Amith – 2420030234, Kotte Manoj – 2420030245, BVS Jayadeep – 2420030381** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

_____	_____	_____
Signature of the Guide	Course Coordinator	Head of the Department
Dr. Bonagani Prathusha	Dr. N. Sunanda	Dr. P. Venkateswara Rao
Department of CSE	Department of CSE	Department of CSE

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. N. Sunanda** and my guide **Dr. Bonagani Prathusha**, Department of Computer Science and Engineering which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P. Venkateswar Rao**, Head of the ***Department of Computer Science and Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Yakkala Guna Veera Sri Amith – 2420030234

Kotte Manoj – 2420030245

BVS Jayadeep – 2420030381

ABSTRACT

Problems with data quality like missing values, repeated records, extreme numeric values, and different text formats can decrease the reliability of reports and analyses to be done later on.

The project “DataCleanAI: Automated Data Quality, Cleaning & Validation Platform” describes a simple web platform that lets users check data and make them good, using one workflow. The developed prototype accepts and analyses CSV and Excel files, presents columns in profile form, shows missing values and identical records, visualizes numeric outliers based on the interquartile range, gives a score of data quality, and applies cleaning procedures.

The cleaning processes include median for numeric types of information, mode to be used for categories, elimination of rows with empty data, deletion of same rows, elimination of outliers based on IQR, and normalization of words.

A Streamlit interface presents statistics, reports on errors and comparisons. Working principles of the platform show users the transformations but not mysterious automatic repairs.

The application is developed using Python, Streamlit, Pandas, NumPy and Plotly. No numerical benchmark results are reported because a controlled experiment has not yet been completed. The quality score is a heuristic, and the cleaning operations should be reviewed in the context of the data.

Keywords: Data Quality, Data Cleaning, Data Profiling, Missing-Value Imputation, Duplicate Detection, Outlier Detection, Streamlit, Data Validation.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	4
		Abstract	5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users/Audience	1
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	5
3	Technology Stack	3.1 Front-end – Streamlit 3.2 Application Logic – Python 3.3 Data Processing – Pandas 3.4 Numerical Operations – NumPy 3.5 Data Visualization – Plotly 3.6 Testing and Output – Python Test Modules and CSV Export	7
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of System Modules 4.3 Application Execution Architecture	9
5	Design	5.1 User Interface Design and Screenshots 5.2 System Workflow / Data Flow Diagram	11
6	Implementation	6.1 Module-wise Implementation 6.2 File Upload and Data Loading 6.3 Data Profiling and Missing-Value Handling 6.4 Duplicate and Outlier Detection 6.5 Heuristic Quality Score 6.6 Cleaning, Reporting and Export	13
7	Features	7.1 List of Main Features 7.2 Working of Each Feature 7.3 File Upload and Sample Data 7.4 Statistics, Issue Reports and Comparison	17
8	Testing	8.1 Functional Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results 8.5 Evaluation of Results	20
9	Deployment	9.1 Steps to Run the Application 9.2 Python Environment Configuration 9.3 Local Application Execution	24
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations	26

11	Future Enhancements	11.1 Planned Features 11.2 Advanced Anomaly Detection and Repair 11.3 Operational Improvements and Evaluation	28
12	Conclusion	12.1 Summary of the Project 12.2 Achievements 12.3 Skills Learned During Development	30
13	References	Research Papers and Technical Documentation	32
14	Appendices	Application Screenshots Sample Code Snippets Installation and Setup Instructions User Manual	33

CHAPTER 1

Introduction

1.1 Background of the Project

Any application leveraging data analytics relies on both the effectiveness of analytical techniques employed and the quality of the information provided. Data from the real world may include missing values, duplicate entries, different naming conventions, non-logical figures, and dissimilar interpretations of the same category.

Implemented faults may be transmuted into dashboards, simple statistics, and machine learning processes. Due to this reason, the process of data cleansing has been known to be of vital importance in data processing [1], [2].

Data quality is multi-dimensional. Completeness and uniqueness of data can be used, but data quality also consists of validity, consistency, relevance and context [3]. As a result of this some cleaning action can lead to improvement of some data aspect but will alter another at the same time.

For example, replacing a missing number with the average value of the whole column allows data to be analyzed easier but primarily hides the fact that number in question was not measured. Similarly, an outlier may either be regarded as an anomaly or an exceptional observation.

DataCleanAI is designed as a prototype for a project that provides a web-based approach to carrying out various data-preparation tasks. A file such as CSV or Excel is first uploaded by the user to examine profile or summary of issues, and thereafter, appropriate cleaning actions can be selected before the results are exported.

1.2 Problem Statement

The problem that is being resolved is the amount of work and uncertainty when preparing tabular data as the user must manually check different types of data errors and fulfill separate transformations.

The desired initial platform should recognize typical errors, determine which operations should be used, keep the original version of data, and get the proper cleaned data. It should also not suggest that there exists one universal rule that can establish the real meaning of each 'suspicious' cell.

The gap at the project level being addressed in this project is practical integration: simple profiling, issues summary, the selected repairs, before-and-after comparison, and export are usually performed by different scripts or tools. DataCleanAI brings all of these activities together in one simple interface, which is an innovative solution; however, no claim is made that the data-cleaning techniques used in this project are original.

1.3 Scope of the Project

The scope of this project includes:

1. Uploading tabular data in CSV and Excel format through a web interface.
2. Profiling the dataset: dimensions, data types, missing values and descriptive statistics.
3. Detecting exact duplicate rows.
4. Detecting numeric outliers using the interquartile range (IQR).
5. Calculating a heuristic data quality score between 0 and 100.
6. Handling missing values using median / mode imputation or deletion of incomplete rows.
7. Removing duplicates, capping outliers and standardising text.
8. Showing the original and cleaned data with interactive Plotly visualizations.
9. Exporting the cleaned dataset in CSV format.

The system is designed for educational purposes and research work. Its functionality does not represent any area-related business rules; therefore, the reliability of detected anomalies and the appropriateness of missing data values cannot be guaranteed.

1.4 Objectives

The main objectives of this project are:

1. To input tabular data in CSV and Excel format through a user-friendly web interface.
2. To give an overview of dataset dimensions, types, number of missing values, duplicated lines, and numeric outliers.
3. To provide a meaningful heuristic quality score with indicators for each component.
4. To allow the user to make decisions on imputation, lines deletion, duplicates deletion, outlier capping, and text-standardizing.
5. To show the cleaned data and previous and future data to make transformations reviewable.
6. To export the cleaned dataset in CSV format and allow for further verification and validation of the work done.

1.5 Importance of the Application

The focus of this system is in making the errors known and the changes made visible so that users may proceed with appropriate actions.

The idea is not to promote the system as a new learning model or a substitute for domain-specific assessment approaches.

Some of the contributions of the project may be outlined as follows: (1) unified interface integrating profiling, issue detection and cleaning; (2) a clear heuristic scores based on complete information, uniqueness, validity and consistency; (3) customizable approach to traditional cleaning techniques; and (4) differentiation

in planned validation of functional assessment and additional verification of repairs made concerning the dataset.

1.6 Target Users/Audience

The intended users of the application include:

1. Students learning data science and data preparation.
2. Faculty members and academic researchers.
3. Developers and analysts who need quick profiling and cleaning of tabular data.
4. Educational institutions demonstrating data-quality and data-cleaning concepts.

CHAPTER 2

System Requirements

2.1 Hardware Requirements

The following hardware configuration is suitable for running the project:

```
Processor       : Intel Core i3 / i5 or equivalent
RAM             : 4 GB minimum, 8 GB recommended
Storage         : At least 1 GB of free space
Operating Input : Keyboard and Mouse
Display         : Standard Laptop or Desktop Display
GPU             : Optional
```

2.2 Software Requirements

The software requirements used for the project are:

1. Operating System: Windows 10 or Windows 11.
2. Programming Language: Python 3.x.
3. Development Environment: Visual Studio Code / Jupyter Notebook.
4. Application Framework: Streamlit.
5. Data Processing: Pandas.
6. Numerical Processing: NumPy.
7. Data Visualization: Plotly.
8. Supported File Formats: CSV and Excel (.xlsx).

2.3 Development Tools and Frameworks

Python:

Python is the main programming language used in this project. It is used for data processing, profiling, quality scoring, cleaning, duplicate detection, outlier detection and file utilities.

Streamlit:

Streamlit is used as the platform for the interface. It handles file uploading, user operations and interactions, and presents statistics, reports on errors and comparisons.

Pandas:

Pandas provides reading of data frames, profiling, imputation, along with representation as well as processing of duplicates.

NumPy:

NumPy helps in performing mathematical computation, such as numeric column selection and related array operations.

Plotly:

Plotly is engaged in creating interactive charts and before/after visual summaries.

CHAPTER 3

Technology Stack

3.1 Front-end – Streamlit

Streamlit is an open-source Python framework used to build interactive web applications.

In this project, Streamlit is used as the platform for the interface. It handles file uploading, user operations and interactions.

The user interface groups the high-level functions in terms of tabs: overview, profiling, issues, cleaning, visualizations, and export.

3.2 Application Logic – Python

Python is used for data processing and for the main application logic of the project.

The code is modularized in terms of the functions for profiling, quality scoring, cleaning, duplicate detection, as well as outlier detection and file utilities.

3.3 Data Processing – Pandas

Pandas provides reading of data frames, profiling, imputation, along with representation as well as processing of duplicates.

It is used for table manipulation throughout the processing part of the prototype.

3.4 Numerical Operations – NumPy

NumPy helps in performing mathematical computation.

In this project it supports numeric column selection and related array operations.

3.5 Data Visualization – Plotly

Plotly is engaged in creating interactive charts.

It is used for interactive charts and before/after visual summaries on the dashboard.

3.6 Testing and Output – Python Test Modules and CSV Export

Python test modules are used for unit checks of the quality scoring and cleaning routines.

The final table receives a clean row index and can be exported as CSV.

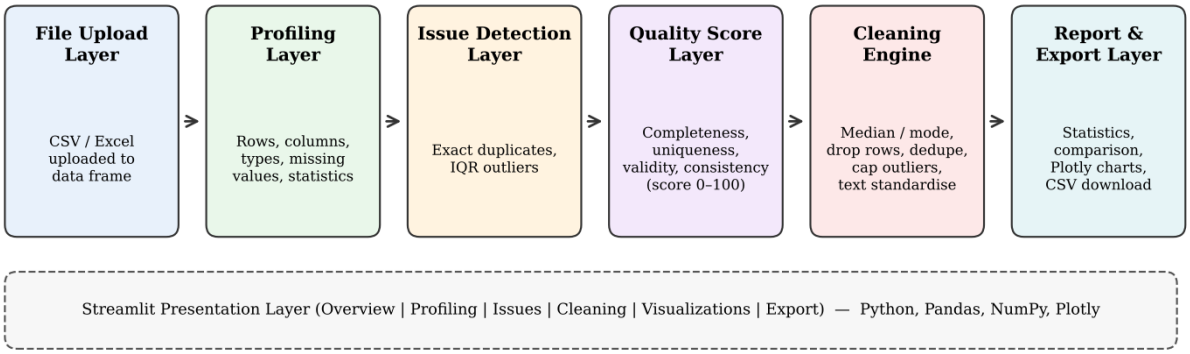
CHAPTER 4

System Architecture

4.1 High-Level Architecture Diagram

The system architecture of DataCleanAI is shown below.

The architecture follows this sequence:



4.2 Description of System Modules

1. File Upload Layer

This layer allows the user to upload a file in CSV or Excel format. The file is processed into a tabular format and kept as a data frame in memory.

2. Profiling Layer

A profiler generates basic structure summaries and calculates missing-value records.

3. Issue Detection Layer

Separate algorithms perform the search for exact duplicates in the dataset and possible anomalies in numeric columns.

4. Quality Score and Cleaning Layer

The score-related algorithm evaluates the required criteria that result in a final score between 0 and 100. The user marks the required options needed for cleansing, and the software produces a data-cleaned output.

5. Presentation and Export Layer

The system delivers a report on the performed tasks, presents the results through the interface, and makes the cleansed data available for downloading.

4.3 Application Execution Architecture

The prototype employs Python for data processing, along with Pandas and NumPy for table manipulation. Streamlit is used as the platform for the interface and Plotly for developing the interactive graphs.

The model consists of two components including a presentation layer and reusable data processor functions. The interface takes care of file uploading, user operations and interactions, while the processing part of the prototype contains modules for data discovery, classification, cleaning and duplicate removal.

The uploaded original table is used as a baseline comparison in the current session, while the cleaning operation is based on its copy. The separation allows for a repetitive review within the session. The prototype is not defined as a permanent multi-user service, and some deployment-related aspects like authentication, storage of the database, and audit logs implementation have to be done explicitly.

CHAPTER 5

Design

5.1 User Interface Design and Screenshots

The application is designed with a simple and interactive interface.

The main interface contains the following sections:

1. Overview

Displays the uploaded dataset summary and the heuristic quality score.

2. Profiling

Displays the number of rows and columns, data types, missing values and descriptive statistics.

3. Issues

Displays missing cells, exact duplicate rows and potential numeric outliers detected using the IQR rule.

4. Cleaning

Allows users to select imputation, row deletion, duplicate removal, outlier capping and text-standardizing options.

5. Visualizations

Displays interactive Plotly charts and before/after comparisons.

6. Export

Allows users to download the cleaned dataset in CSV format.

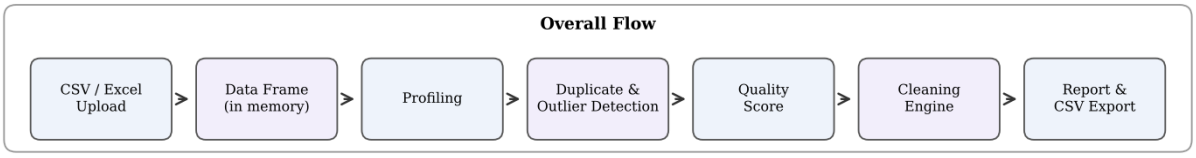
Insert the screenshots of the actual working application under this section.

5.2 System Workflow / Data Flow Diagram

A Data Flow Diagram represents how data moves through the system.

In this project, the file provided by the user is read into a data frame and passed to the profiling and issue detection modules. After the quality score is calculated, the selected cleaning options are applied to a copy of the data, and the results are displayed and exported.

The overall data flow is:



The detailed data flow consists of the following steps:

1. CSV/Excel uploaded → file reader → data frame in the memory.
2. Data frame → profiling system → numbers, summaries of columns and information about missing data.
3. Data frame → duplicate detection system and outlier detection system → summary.
4. Profiling and summary data → the function for calculating quality score of data.
5. Initial data frame + options selected by user → cleaning process is done by the engine.
6. Raw report + result of cleaning process → presentation layer and visualization of data.

CHAPTER 6

Implementation

6.1 Module-wise Implementation

The project is divided into four major modules.

Module 1: File Upload and Profiling Module

This module accepts CSV and Excel files and generates a structural profile of the dataset, including rows, columns, data types, missing values and descriptive statistics.

Input: CSV / Excel file.

Output: Data frame and dataset profile.

Module 2: Issue Detection Module

This module reports missing cells, exact duplicate rows and potential numeric outliers based on the interquartile range.

Input: Data frame.

Output: Missing-value, duplicate and outlier summaries.

Module 3: Quality Scoring and Cleaning Module

This module calculates the heuristic quality score and applies the cleaning operations selected by the user.

Input: Profile, issue summaries and user-selected options.

Output: Quality score, cleaned data and action counts.

Module 4: Visualization and Export Module

This module displays the statistics, reports on errors and comparisons, and makes the cleaned data available for download.

Input: Original data, cleaned data and cleaning report.

Output: Interactive charts, comparison views and CSV file.

6.2 File Upload and Data Loading

The application accepts uploads of CSV and Excel files and provides a sample data for the client for demonstration.

After the file is received, Pandas reads it into a data frame. The uploaded original table is used as a baseline comparison in the current session, while the cleaning operation is based on its copy.

6.3 Data Profiling and Missing-Value Handling

Data profiling calculates and provides data such as number of rows and columns regarding the columns, missing values, data types and some obtained descriptive statistics. It serves as a descriptive measure as it determines the places where inspection is required but not whether its value is correct or not.

For missing values, the user is given the option to choose between the median/mode imputation and deleting the incomplete rows. In the case of numeric column median imputation is used. For non-numeric columns the mode (the most frequent value) is used.

Although median imputation is more robust than mean imputation, it can still distort the relationship and distribution. Mode imputation may lead to overestimation of the sizes of certain categories, while deleting incomplete records may result in losing useful information.

6.4 Duplicate and Outlier Detection

Exact duplicate detection relies on row equality throughout all of its columns. It is useful for identifying repeated rows but is not qualified enough to detect near-duplicates, such as spelling variations or slightly different addresses. For these purposes record-linkage methods and similarity metrics are used.

For each variable with numerical data, the prototype calculates Q1 - first quartile, Q3 - third quartile, and the interquartile range IQR = Q3 - Q1. The lower threshold is Q1 - 1.5 × IQR and the upper threshold is Q3 + 1.5 × IQR. The values falling below and above these thresholds are treated as outliers. If a capping option is chosen, then, the values are clipped.

This is a general exploratory rule introduced by Tukey [6], it does not represent a wide acceptance of defining an outlier. It may not be appropriate for data with high skewness, small sample sizes, or other naturally heavy-tailed variables.

6.5 Heuristic Quality Score

The four aspects calculated are completeness, uniqueness, validity and consistency. The total number of cells is represented by N, and the number of missing cells is represented by M. If D represents the ratio of duplicate rows to all rows, the scores are calculated as follows:

```
C = 100 × (1 − M/N)
U = 100 × (1 − D)
S = (C + U) / 2
Q = 0.35C + 0.25U + 0.20V + 0.20S
```

The validity component V applies to a basic numeric-value check; in its current form, it is limited and should not be taken as a complete type of schema or domain validation.

The score is rounded and bounded to the 0–100 range. The weights are design choices, not weights estimated from expert labels or empirical outcomes. The score is therefore a comparative heuristic for the selected indicators and must not be described as a probability that the dataset is correct.

6.6 Cleaning, Reporting and Export

For text-like columns, the prototype eliminates unnecessary leading and trailing spaces, merges repeated spaces into one space, and converts the text into title

case. Although this can mitigate the differences in formatting somewhat, it can affect the way certain product codes, email, or case-sensitive identifiers look, thus it is advised to omit those columns from the process of standardisation in case the cases have a meaning.

The records contain information on the number of values completed, the number of skipped rows due to missingness, eliminated duplicates, modified numeric outlier values, and standardized text columns. The final table receives a clean row index and can be exported as CSV.

The production-level system should also maintain reversible processing logs and verify output schema and conditions before export.

CHAPTER 7

Features

7.1 List of Main Features

The major features of the application are:

1. CSV and Excel file upload.
2. Dataset profiling.
3. Missing-value report.
4. Exact duplicate detection.
5. IQR-based outlier detection.
6. Heuristic quality score from 0 to 100.
7. Median/mode imputation and incomplete-row deletion.
8. Duplicate removal and outlier capping.
9. Text standardisation.
10. Before/after comparison and interactive visualizations.
11. Cleaning report and CSV export.

7.2 Working of Each Feature

File Upload:

The user uploads a CSV or Excel file. The application reads it into a data frame in memory.

Data Profiling:

The profiler provides the number of rows and columns, data types, missing values and descriptive statistics.

Missing-Value Handling:

The user chooses between median/mode imputation and deleting incomplete rows.

Duplicate Detection and Removal:

Exact duplicate rows are identified by row equality across all columns and can be removed.

Outlier Detection and Capping:

Values below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR are flagged and, if the capping option is chosen, are clipped to the thresholds.

Text Standardisation:

Leading and trailing spaces are removed, repeated spaces are merged and the text is converted into title case.

Quality Score:

The application combines completeness, uniqueness, validity and consistency into a heuristic score between 0 and 100.

Cleaning Report and Export:

The application reports the number of actions performed and allows the cleaned dataset to be downloaded as a CSV file.

7.3 File Upload and Sample Data

The application accepts uploads of CSV and Excel files and provides a sample data for the client for demonstration.

The parameters for cleaning are chosen explicitly by the user and are not applied automatically.

7.4 Statistics, Issue Reports and Comparison

A Streamlit interface presents statistics, reports on errors and comparisons.

The dashboard shows the original and cleaned data so that the transformations remain reviewable. Working principles of the platform show users the transformations but not mysterious automatic repairs.

CHAPTER 8

Testing

8.1 Functional Testing

Functional testing is performed to verify whether the main features of the application work as expected.

The following operations are tested:

1. Uploading a CSV file.
2. Uploading an Excel file.
3. Viewing the dataset profile.
4. Detecting missing values, duplicates and outliers.
5. Calculating the quality score.
6. Applying the selected cleaning options.
7. Viewing the before/after comparison.
8. Exporting the cleaned CSV file.

8.2 Integration Testing

Integration testing checks whether the different modules work together correctly.

The following sequence is verified:

1. User uploads a file.
2. The file reader converts it into a data frame.
3. The profiling and detection modules receive the data frame.
4. The quality score is calculated.
5. The cleaning engine applies the options selected by the user.
6. The results are displayed and the cleaned data is exported.

8.3 Tools Used for Testing

The following tools and libraries are used:

- Python test modules for unit checks of quality scoring and cleaning routines.
- Streamlit for testing the user interface.
- Pandas for data handling.
- NumPy for numerical operations.
- Plotly for visualizing results.
- VS Code and Jupyter Notebook for development and testing.

Postman testing is not applicable because the current implementation does not use a separate REST backend.

8.4 Test Cases and Results

Test Case ID	Test Case	Expected Result	Status
TC-01	Upload a CSV file	File should be read and the dataset summary displayed	Pass (if successfully tested)
TC-02	Upload an Excel file	File should be read and the dataset summary displayed	Pass (if successfully tested)
TC-03	Detect duplicates and outliers	Exact duplicates and IQR-based potential outliers should be reported	Pass (if successfully tested)
TC-04	Calculate the quality score	Score between 0 and 100 with its indicators should be displayed	Pass (if successfully tested)
TC-05	Apply cleaning and export	Cleaned data and action counts should be shown and the CSV downloadable	Pass (if successfully tested)

8.5 Evaluation of Results

The present deliverable is both a working prototype and a base for code, not a completed benchmarking study.

The process employed is set to generate four results: (1) a structural profile for the uploaded data; (2) issue summaries for missing cells, accurate duplicates, and IQR based outliers; (3) a heuristic score of quality with its indicators; and (4) a clean table with number of transformations. The table below provides a summary of the outputs and interpretational limits associated with these outputs.

Module	Output	Interpretation Limit
Profiling	Dataset summary and column profile	Descriptive, not a correctness audit
Duplicate analysis	Duplicate row count and details	Near-duplicates are not detected
Outlier analysis	Potential outlier counts by column	Rare valid values may be flagged
Quality scoring	Heuristic score from 0 to 100	Not calibrated against expert labels
Cleaning	Cleaned data and action counts	Repairs need contextual review
Export	CSV file	Schema and business rules need validation

The flags only denote issues that need to be validated, not established truth. Similarly, a higher score after cleaning suggests the improvement of the chosen indicators but does not guarantee the correctness of repair or adequacy of the data for downstream operations.

No numerical benchmark results are reported here because a controlled experiment with a defined dataset, known injected errors, baseline methods, and repeated measurements has not been completed. Reporting an invented accuracy percentage, processing-time reduction, or superiority claim would be misleading.

A suitable evaluation would compare the original and cleaned tables using precision and recall for defect detection, precision of repairs against known ground truth, number of altered cells, runtime, and memory consumption.

CHAPTER 9

Deployment

9.1 Steps to Run the Application

The application can be executed locally using the following steps:

1. Install Python on the computer.
2. Open the project folder in Visual Studio Code.
3. Create a Python virtual environment.
4. Activate the virtual environment.
5. Install the required Python libraries.
6. Run the Streamlit application.

Command to execute the application:

```
python -m streamlit run app.py
```

After executing the command, Streamlit displays a local URL in the terminal.

Open that URL in a web browser to access the application.

9.2 Python Environment Configuration

The project uses a Python virtual environment to manage the required libraries.

The environment contains the necessary packages for data processing, visualization and the Streamlit interface: Pandas, NumPy, Plotly and Streamlit.

9.3 Local Application Execution

The application runs locally through Streamlit.

The prototype is not defined as a permanent multi-user service, and some deployment-related aspects like authentication, storage of the database, and audit logs implementation have to be done explicitly.

CHAPTER 10

Challenges and Limitations

10.1 Issues Faced During Development

During the development of this project, we faced several challenges.

Multiple Quality Dimensions:

Data quality is multi-dimensional, and some cleaning action can lead to improvement of one data aspect but alter another at the same time.

Missing Values:

Replacing a missing number with the average value of the whole column makes the data easier to analyse but hides the fact that the number was not measured.

Outliers:

An outlier may either be regarded as an anomaly or an exceptional observation, so the IQR rule cannot decide which values are truly wrong.

Text Standardisation:

Title-case normalisation can damage acronyms, identifiers and values for which case is meaningful.

Near-Duplicate Records:

Exact duplicate detection does not resolve fuzzy or entity-level duplicates such as spelling variations.

10.2 Solutions Applied

The following solutions were applied:

1. The parameters for cleaning are chosen explicitly by the user and are not applied automatically.
2. The uploaded original table is kept as a baseline while cleaning works on a copy.
3. Before/after comparison makes every transformation visible.
4. A cleaning report records the number of actions performed.
5. The heuristic score is shown with its separate indicators.
6. Outlier capping is optional and the interquartile range rule is documented as an exploratory rule.
7. Users are advised to omit case-sensitive columns from text standardisation.

10.3 Current Limitations

The current system has the following limitations:

1. The quality score uses manually chosen weights and a limited set of indicators; it is not empirically calibrated.
2. The current validity calculation is a basic heuristic and does not check arbitrary schemas, allowed ranges, foreign keys, or domain constraints.
3. Exact duplicate detection does not resolve fuzzy or entity-level duplicates.
4. Median/mode imputation may bias distributions and relationships; row deletion may cause data loss.
5. IQR-based outlier capping can suppress meaningful extreme observations and is not appropriate for every distribution.
6. Title-case normalization can damage acronyms, identifiers, and values for which case is meaningful.
7. The prototype does not yet provide comprehensive audit trails, role-based access, database persistence, or enterprise privacy controls.
8. No controlled empirical evaluation has yet established detection precision, repair accuracy, scalability, or user benefits.

CHAPTER 11

Future Enhancements

11.1 Planned Features

The project can be extended in several ways.

Reproducible Evaluation Suite:

Future development should prioritize a reproducible evaluation suite with injected missingness, duplicates, text variations, invalid values, and known numeric anomalies.

Configurable Schema Rules:

Configurable schema rules, range and type validation, date parsing and category standardization dictionaries can be added.

Fuzzy Duplicate Matching:

Fuzzy duplicate matching can detect near-duplicate records.

Reusable Cleaning Recipes:

Support for reusable cleaning recipes can be introduced.

Reversible Transformation History:

A reversible transformation history and downloadable cleaning reports can be provided.

11.2 Advanced Anomaly Detection and Repair

Isolation Forest or other anomaly-detection methods can be added. Advanced repair approaches could combine integrity constraints and statistical evidence, as explored by systems such as HoloClean [5].

Such extensions should be optional and should expose confidence or review requirements rather than silently changing data.

11.3 Operational Improvements and Evaluation

Operational improvements include a reversible transformation history, downloadable cleaning reports, reproducible configuration files, larger-file handling, stronger file validation, accessibility improvements, and privacy-conscious deployment.

Each issue type should have a labeled reference set so detection precision, recall, and repair correctness can be measured. The study should compare the current rule-based approach with simple baselines and report runtime and memory use across increasing dataset sizes.

CHAPTER 12

Conclusion

12.1 Summary of the Project

The project "DataCleanAI: Automated Data Quality, Cleaning & Validation Platform" presents a lightweight web-based platform for automated assistance with tabular data profiling, quality assessment, cleaning, visualization, and export.

The prototype combines missing-value handling, exact duplicate removal, IQR-based outlier screening and capping, text normalization, and a transparent multi-component quality score within a single workflow.

Its main project contribution is the integration of common preparation steps with visible controls and a cleaning report, rather than the invention of a new cleaning algorithm.

The current implementation is best treated as a demonstrator and a foundation for systematic evaluation. The quality score is heuristic, some cleaning operations can alter legitimate information, and no empirical performance claims are made in the absence of a controlled benchmark.

Future work should establish ground-truth test data, measure defect detection and repair quality, expand domain-aware validation, and add auditable, reversible transformations. These steps are necessary before the platform can be recommended for high-stakes or production data workflows.

12.2 Achievements

The major achievements of the project include:

1. Developed a web-based platform for data profiling, quality assessment and cleaning.
2. Implemented CSV and Excel file upload.
3. Implemented missing-value, duplicate and IQR-based outlier detection.
4. Implemented a transparent heuristic quality score.
5. Implemented user-controlled cleaning options and text standardisation.
6. Developed interactive visualizations and a before/after comparison.
7. Implemented a cleaning report and CSV export.

12.3 Skills Learned During Development

Through this project, we gained practical experience in:

1. Python programming.
2. Data profiling and data-quality assessment.
3. Missing-value, duplicate and outlier handling techniques.
4. Pandas and NumPy for data processing.
5. Streamlit application development.
6. Interactive data visualization with Plotly.
7. Unit testing of cleaning and scoring routines.
8. Team collaboration and technical documentation.

CHAPTER 13

References

1. E. Rahm and H. H. Do, “Data Cleaning: Problems and Current Approaches,” IEEE Data Engineering Bulletin, vol. 23, no. 4, pp. 3–13, Dec. 2000. Available: https://old.dbs.uni-leipzig.de/en/publication/title/data_cleaning_problems_and_current_approaches
2. X. Chu, I. F. Ilyas, S. Krishnan, and J. Wang, “Data Cleaning: Overview and Emerging Challenges,” in Proc. ACM SIGMOD International Conference on Management of Data (SIGMOD ’16), 2016, pp. 2201–2206, doi: 10.1145/2882903.2912574.
3. R. Y. Wang and D. M. Strong, “Beyond Accuracy: What Data Quality Means to Data Consumers,” Journal of Management Information Systems, vol. 12, no. 4, pp. 5–33, 1996, doi: 10.1080/07421222.1996.11518099.
4. S. Kandel, A. Paepcke, J. Hellerstein, and J. Heer, “Wrangler: Interactive Visual Specification of Data Transformation Scripts,” in Proc. ACM CHI Conference on Human Factors in Computing Systems, 2011, doi: 10.1145/1978942.1979444.
5. T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, “HoloClean: Holistic Data Repairs with Probabilistic Inference,” Proceedings of the VLDB Endowment, vol. 10, no. 11, pp. 1190–1201, 2017, doi: 10.14778/3137628.3137631.
6. J. W. Tukey, Exploratory Data Analysis. Reading, MA, USA: Addison-Wesley, 1977.
7. Streamlit Documentation. Available at: https://docs.streamlit.io/
8. Pandas Documentation. Available at: https://pandas.pydata.org/docs/
9. NumPy Documentation. Available at: https://numpy.org/doc/
10. Plotly Python Documentation. Available at: https://plotly.com/python/

CHAPTER 14

Appendices

14.1 Application Screenshots

The following screenshots demonstrate the working of the application:

1. File upload page.
2. Overview page with the quality score.
3. Data profiling page.
4. Issues page (missing values, duplicates and outliers).
5. Cleaning options.
6. Before/after comparison.
7. Interactive visualizations.
8. Export of the cleaned dataset.

14.2 Sample Code Snippets

The following is a simplified example of the outlier-capping and quality-score logic described in this report:

```
import pandas as pd

# IQR-based outlier capping for one numeric column
q1 = df[col].quantile(0.25)
q3 = df[col].quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df[col] = df[col].clip(lower, upper)

# Heuristic quality score
quality = 0.35 * C + 0.25 * U + 0.20 * V + 0.20 * S
```

14.3 Installation and Setup Instructions

Step 1: Install Python 3.x.

Step 2: Open the project folder in Visual Studio Code.

Step 3: Activate the virtual environment.

```
.\venv\Scripts\Activate.ps1
```

Step 4: Install the required libraries.

```
pip install -r requirements.txt
```

Step 5: Run the application.

```
python -m streamlit run app.py
```

Step 6: Open the local URL displayed in the terminal.

14.4 User Manual

The application can be used through the following steps:

1. Open the Streamlit application in a browser.
2. Upload a CSV or Excel file, or use the sample data.
3. View the dataset overview and profile.
4. Check the reported missing values, duplicates and outliers.
5. Check the heuristic quality score and its components.
6. Select the required cleaning options.
7. Apply the cleaning operations.
8. Compare the original and cleaned data.
9. Download the cleaned dataset as a CSV file.

14.5 Team Contributions

Yakkala Guna Veera Sri Amith:

- Developed the front-end of the application using Streamlit.

- Worked on the file upload section and the interface tabs (overview, profiling, issues, cleaning, visualizations and export).
- Worked on the Plotly visualizations and the before/after comparison.
- Prepared the project presentation (PPT).

Kotte Manoj:

- Developed the back-end data processing modules.
- Worked on data profiling, missing-value handling and duplicate detection.
- Worked on IQR-based outlier detection and capping, and text standardisation.
- Worked on the cleaning report and CSV export.

BVS Jayadeep:

- Worked on the heuristic quality score (completeness, uniqueness, validity and consistency).
- Wrote the Python test modules and worked on testing and debugging.
- Prepared the project documentation and the research paper.
- Worked on connecting the back-end modules with the front-end.

All three team members contributed to project integration, testing, debugging and documentation.